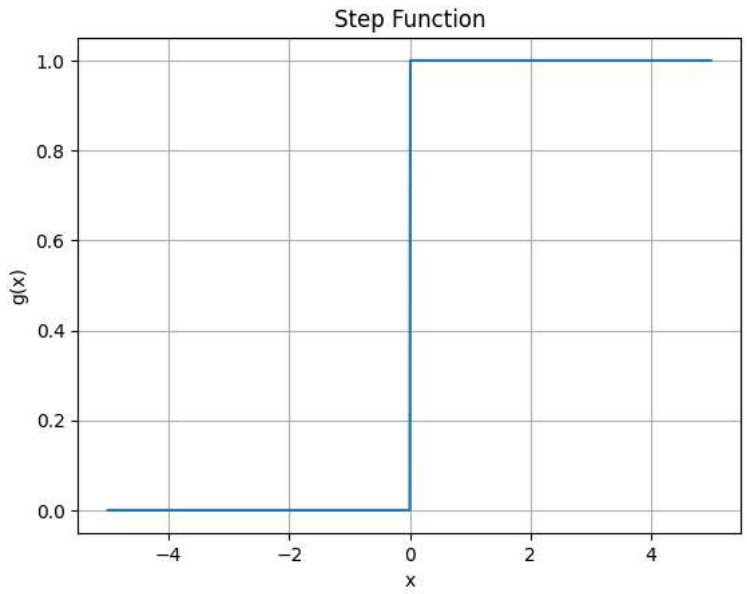
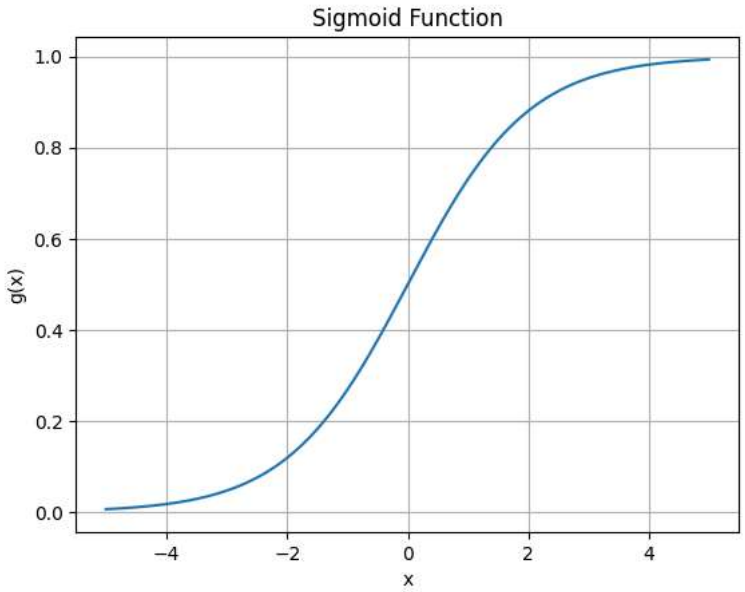
```
#GeLU, ELU, Softplus & SELU were implemented by Jaiden Camp
#Leaky ReLU, PReLU, PReLU, SiLU, & Gaussian implmeted by Jason Stys
# https://colab.research.google.com/drive/1pHcrJj_FO1TKtsydpYIHr7x5xPQr88ed?usp=sharing
```

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
```

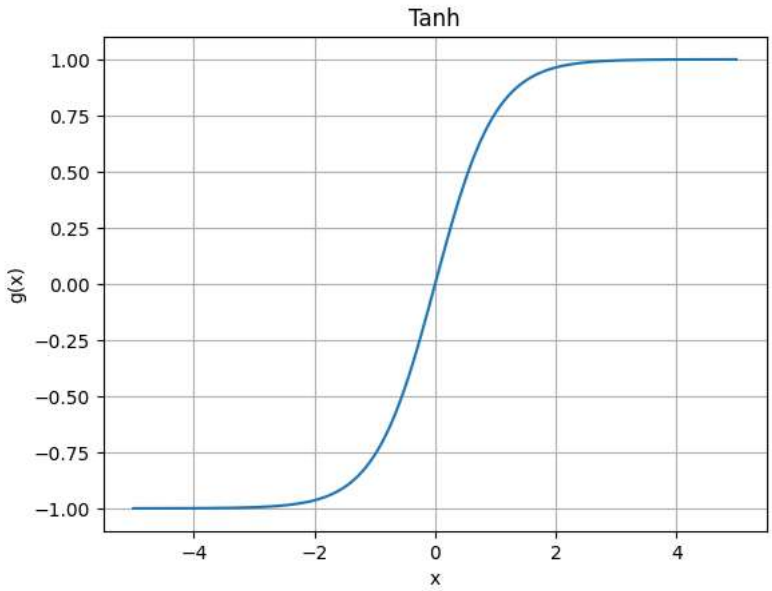
```
x = np.arange(-5, 5, 0.01)
y = np.where(x < 0, 0, 1)

plt.plot(x, y)
plt.title('Step Function')
plt.xlabel('x')
plt.ylabel('g(x)')
plt.grid()
plt.show()
```



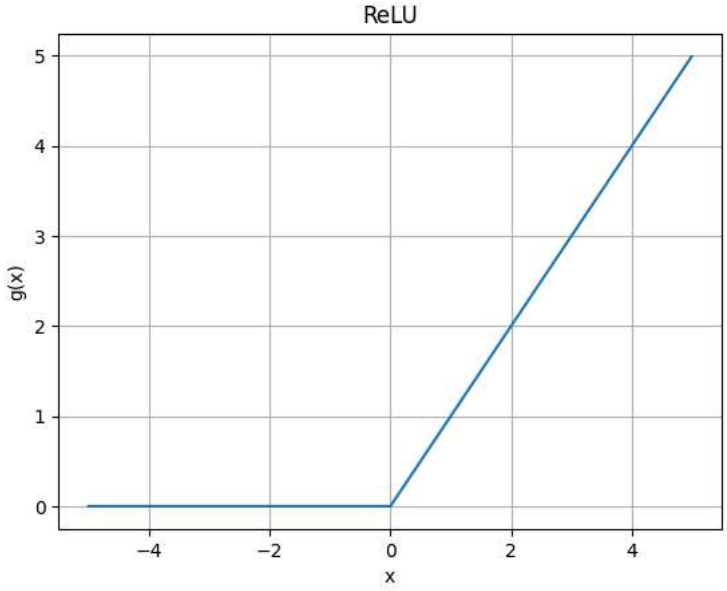

```
def sigmoid(x):
  return 1 / (1 + np.exp(-x))
x = np.arange(-5, 5, 0.01)
plt.plot(x, sigmoid(x))
plt.title('Sigmoid Function')
plt.xlabel('x')
plt.ylabel('g(x)')
plt.grid()
plt.show()
```




```
def tanh(x):
  return (np.exp(x) - np.exp(-x)) / (np.exp(x) + np.exp(-x))

x = np.arange(-5, 5, 0.01)
plt.plot(x, tanh(x))
plt.title('Tanh')
plt.xlabel('x')
plt.ylabel('g(x)');
plt.grid()
plt.show()
```



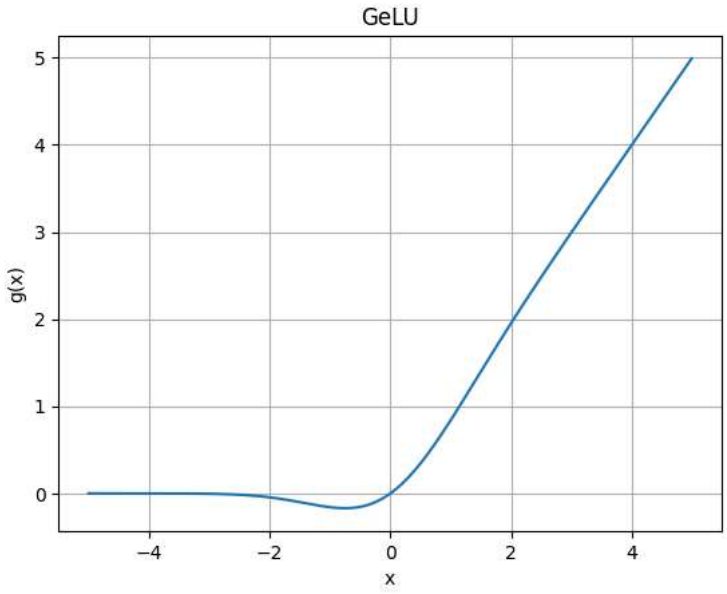

```
x = np.arange(-5, 5, 0.01)
y = np.maximum(0, np.where(x < 0, 0, x))

plt.plot(x, y)
plt.title('ReLU')
plt.xlabel('x')
plt.ylabel('g(x)')
plt.grid()
plt.show()
```




```
x = np.arange(-5, 5, 0.01)
y = np.where(x < 0, 0 ,1)

def GeLU(x):
   return 0.5 * x * (1.0 + tf.math.erf(x / tf.sqrt(2.0)))

plt.plot(x, GeLU(x))
plt.title("GeLU")
plt.xlabel('x')
plt.ylabel('g(x)');
plt.grid()
plt.show
```

matplotlib.pyplot.show
`def show(*args, **kwargs) -> None`

```
Display all open figures.

Parameters
----------
block : bool, optional
    Whether to wait for all figures to be closed before returning.
```

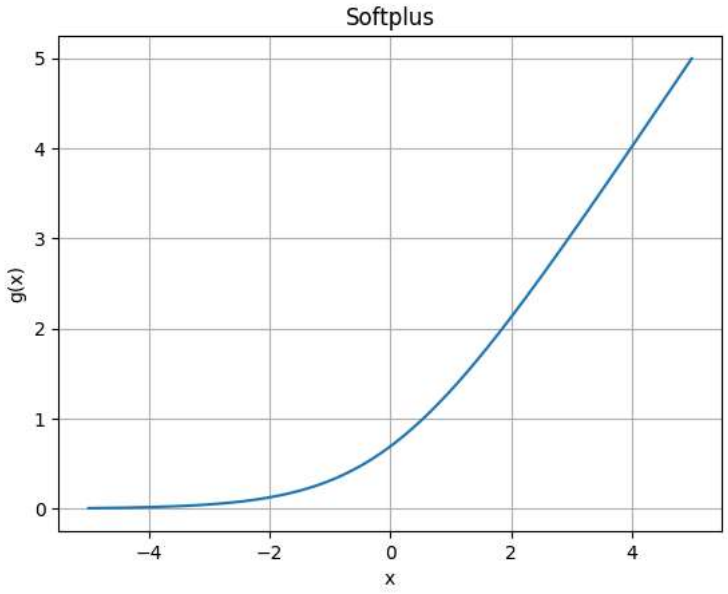



```
x = np.arange(-5, 5, 0.01)
y = np.where(x < 0, 0 ,1)

def Softplus(x):
   return np.log(1 + np.exp(x))

plt.plot(x, Softplus(x))
plt.title("Softplus")
plt.xlabel('x')
```

```
plt.ylabel('g(x)');
plt.grid()
plt.show
```

matplotlib.pyplot.show
```
def show(*args, **kwargs) -> None
```

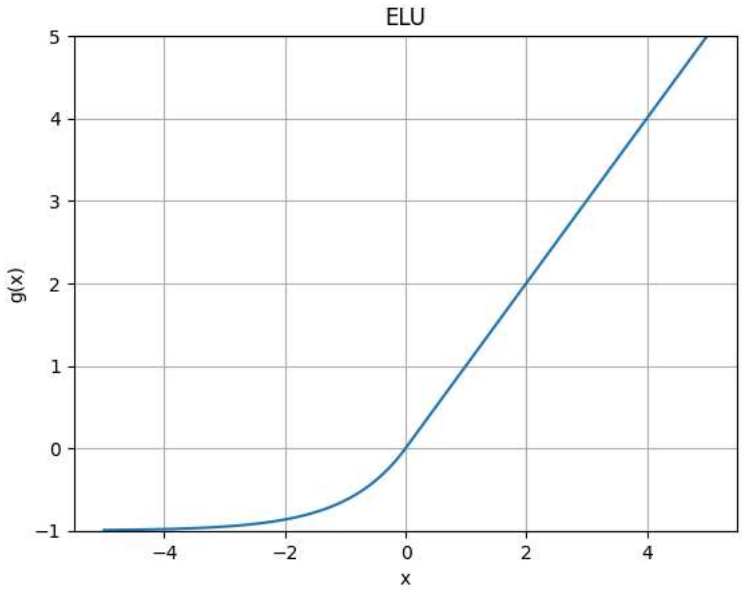
```
Display all open figures.

Parameters
----------
block : bool, optional
    Whether to wait for all figures to be closed before returning.
```




```
def ELU(x):
   return np.where(x > 0, x, (np.exp(x) - 1))

x = np.arange(-5, 5, 0.01)
y = ELU(x)

plt.plot(x, y)
plt.title("ELU")
plt.xlabel('x')
plt.ylabel('g(x)');
plt.grid()
plt.ylim(-1, 5)
plt.show
```

```
matplotlib.pyplot.show
def show(*args, **kwargs) -> None
```

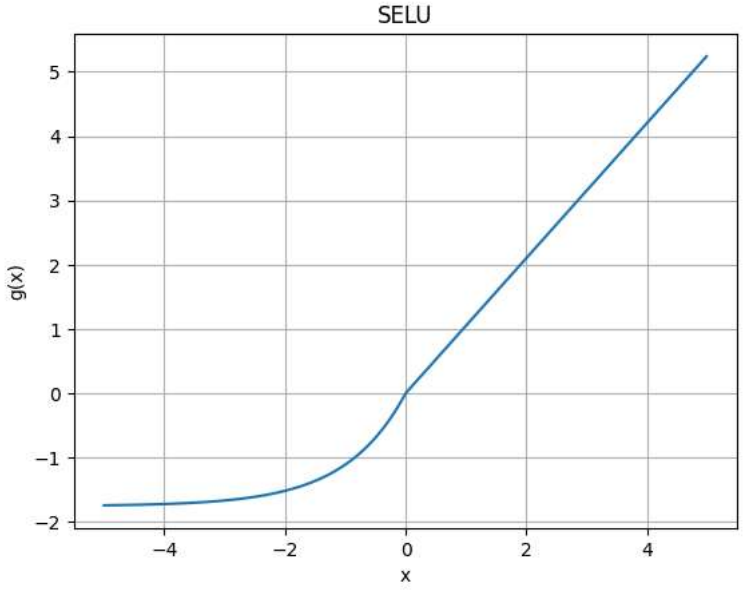
```
Display all open figures.

Parameters
----------
block : bool, optional
    Whether to wait for all figures to be closed before returning.
```



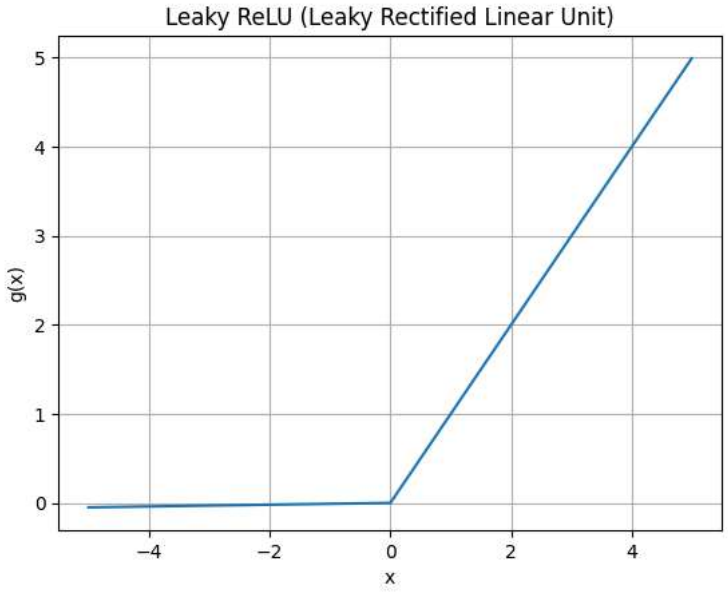

```
x = np.arange(-5, 5, 0.01)
y = np.where(x < 0, 0 ,1)

def SELU(x):
   return np.where(x > 0, 1.05 * x, (1.67 * 1.05) * (np.exp(x) - 1))

plt.plot(x, SELU(x))
plt.title("SELU")
plt.xlabel('x')
plt.ylabel('g(x)');
plt.grid()
plt.show
```

```
matplotlib.pyplot.show
def show(*args, **kwargs) -> None
```

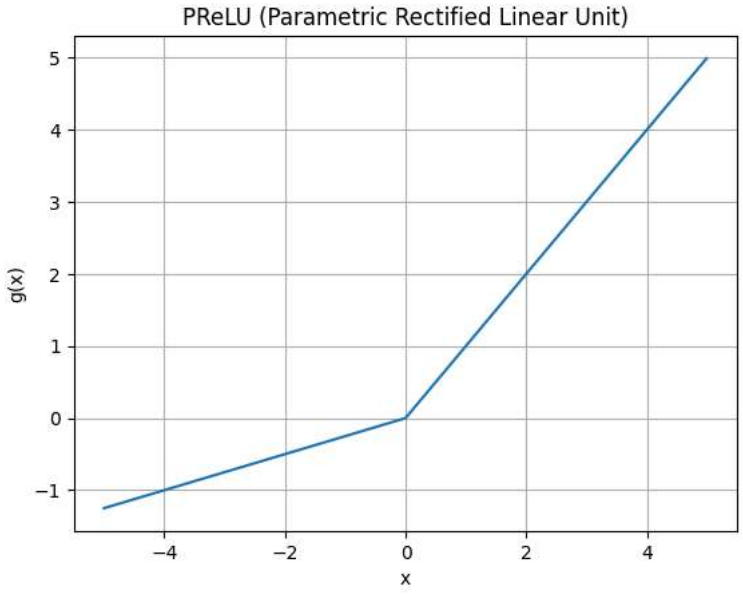
```
Display all open figures.

Parameters
----------
block : bool, optional
    Whether to wait for all figures to be closed before returning.
```



```
# Leaky Rectified Linear Unit (Leaky ReLU)

def leaky_relu(x, alpha=0.01):
  return np.where(x > 0, x, alpha * x)

x = np.arange(-5, 5, 0.01)
y = leaky_relu(x)

plt.plot(x, y)
plt.title('Leaky ReLU (Leaky Rectified Linear Unit)')
plt.xlabel('x')
plt.ylabel('g(x)')
plt.grid()
plt.show()
```




```
# Parametric Rectified Linear Unit (PReLU)

def prelu(x, alpha=0.25):

  return np.where(x > 0, x, alpha * x)

x = np.arange(-5, 5, 0.01)
y = prelu(x)

plt.plot(x, y)
plt.title('PReLU (Parametric Rectified Linear Unit)')
plt.xlabel('x')
plt.ylabel('g(x)')
plt.grid()
plt.show()
```



```
# Sigmoid Linear Unit (SiLU)

def silu(x):
  return x / (1 + np.exp(-x))

x = np.arange(-5, 5, 0.01)
y = silu(x)

plt.plot(x, y)
plt.title('SiLU (Sigmoid Linear Unit)')
plt.xlabel('x')
plt.ylabel('g(x)')
plt.grid()
plt.show()
```